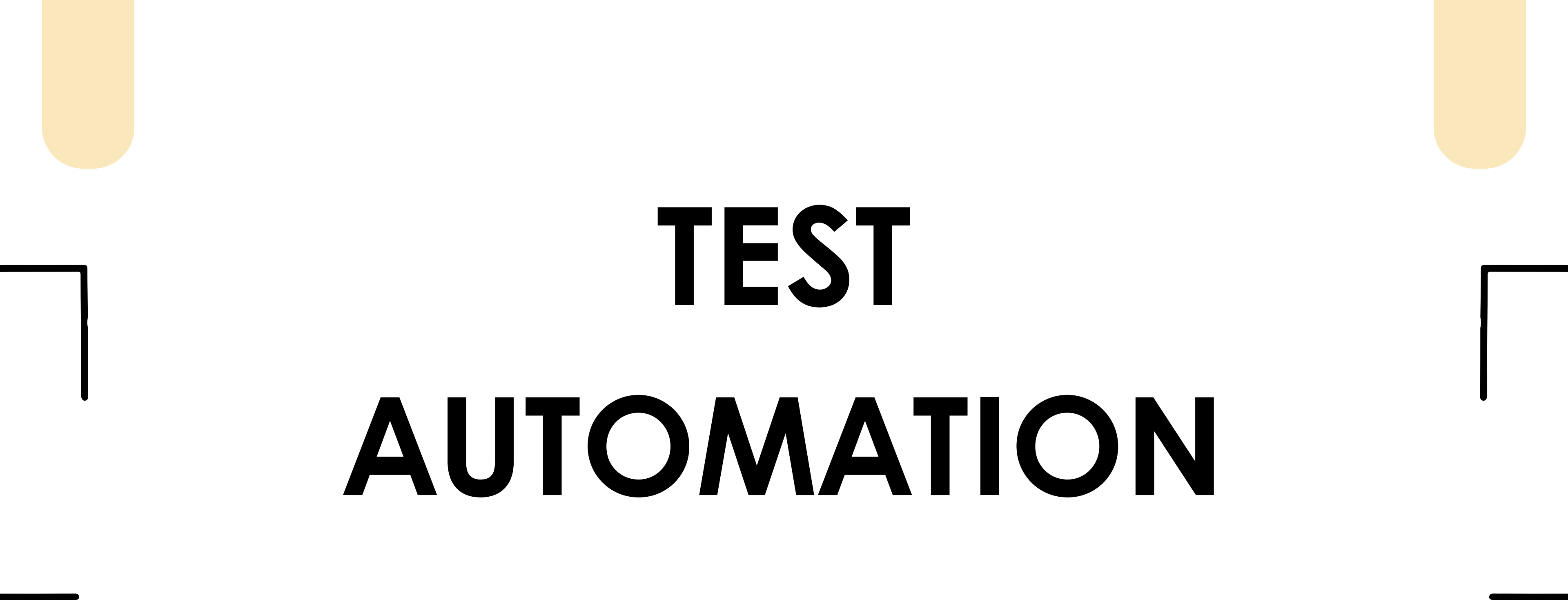




TEST AUTOMATION

WHAT IS TEST AUTOMATION?

- Test automation involves using software tools to test other software automatically.
- It helps automate the execution of repetitive test cases, reducing manual effort.
- Automation improves test coverage and accuracy of testing.
- It enhances the overall quality of the software product.
- Automating tests saves time and effort in the long run, especially for repeated testing cycles.

BENEFITS OF TEST AUTOMATION

- Automation helps in immediate testing
- Automation can protect an organization against attrition of test engineers
- Test automation opens up opportunities for better utilization of global resources
- Test cases for certain types of testing such as reliability testing, stress testing, load and performance testing, cannot be executed without automation
- Automation saves time as software can execute test cases faster than human do
- Test automation can free the test engineers from mundane tasks and make them focus on more creative tasks
- Automated tests can be more reliable

SCOPE OF TEST AUTOMATION

(WHAT TO AUTOMATE)

I. Identifying Testing Types Amenable to Automation

Certain categories of testing are natural candidates for automation due to their repetitive or resource-intensive nature:

- **Stress, Reliability, Scalability, and Performance Testing:** These require running tests on many machines for extended periods (e.g., 24–48 hours). Automation is essential here as it is nearly impossible to find human testers to perform these repetitive tasks consistently.
- **Regression Tests:** Because these are executed multiple times during every product development phase, automation saves significant time and effort, allowing testers to focus on creative tasks like ad hoc testing.
- **Functional Tests:** These often require complex setups and specialized expertise. Automating them once allows less-skilled personnel to run them regularly.
- **The "Rule of Thumb":** Automation should be considered if a test case needs to be executed at least 10 times in the near future (e.g., one year) and the effort to automate it does not exceed 10 times the effort of a single manual execution.

II. Automating Areas Less Prone to Change

To minimize maintenance and rework of scripts, focus on stable areas:

- **Stable Requirements:** Target basic functionalities (like steering/brakes in a car) that rarely change. These form the regression test bed for daily builds and smoke tests.
- **Backend vs. GUI:** User Interfaces (UI) change frequently. Automation should prioritize non-GUI or backend elements first.
- **Pluggability:** Maintain a clear demarcation between UI and non-UI automation so that backend scripts remain reusable even if the interface is redesigned.

III. Automating for Standards and Certification

- **Compliance to Standards:** Tests pertaining to industry standards (e.g., JDBC interfaces) are ideal because they undergo minimal change and provide backward compatibility.
- **Market Opportunity:** Test suites developed for standards can often be sold as independent commercial tools.
- **Certification Testing:** Hardware/software compatibility testing (ensuring a product works on new hardware) requires perfectly compliant results every time. Automation is the most effective way to handle these certification suites.

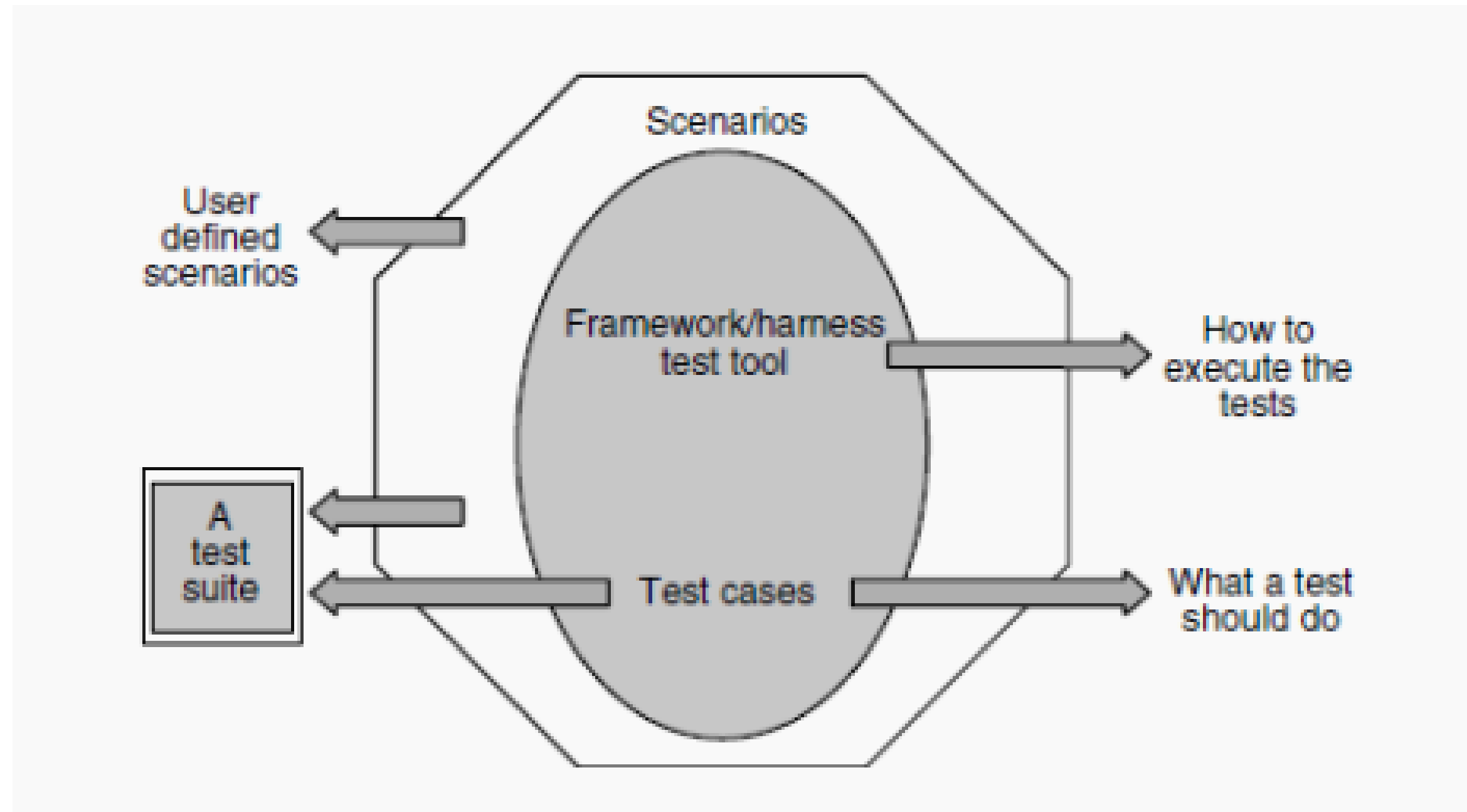
IV. Management and Strategic Aspects

Successful automation requires more than just technical skill; it requires administrative alignment:

- **Management Commitment:** Since automation is an ongoing effort that lasts until product obsolescence, it requires phased permissions and sustained funding.
- **Return on Investment (ROI):** Focus on areas where automation covers many permutations with minimal code. Prioritize tests that are easy to automate and have clear expected results to demonstrate quick returns to management.
- **Prioritization (Stephen Covey's Principle):** Apply the First Things First principle. Categorize test cases as High, Medium, or Low priority based on customer expectations, and automate from High priority downwards.

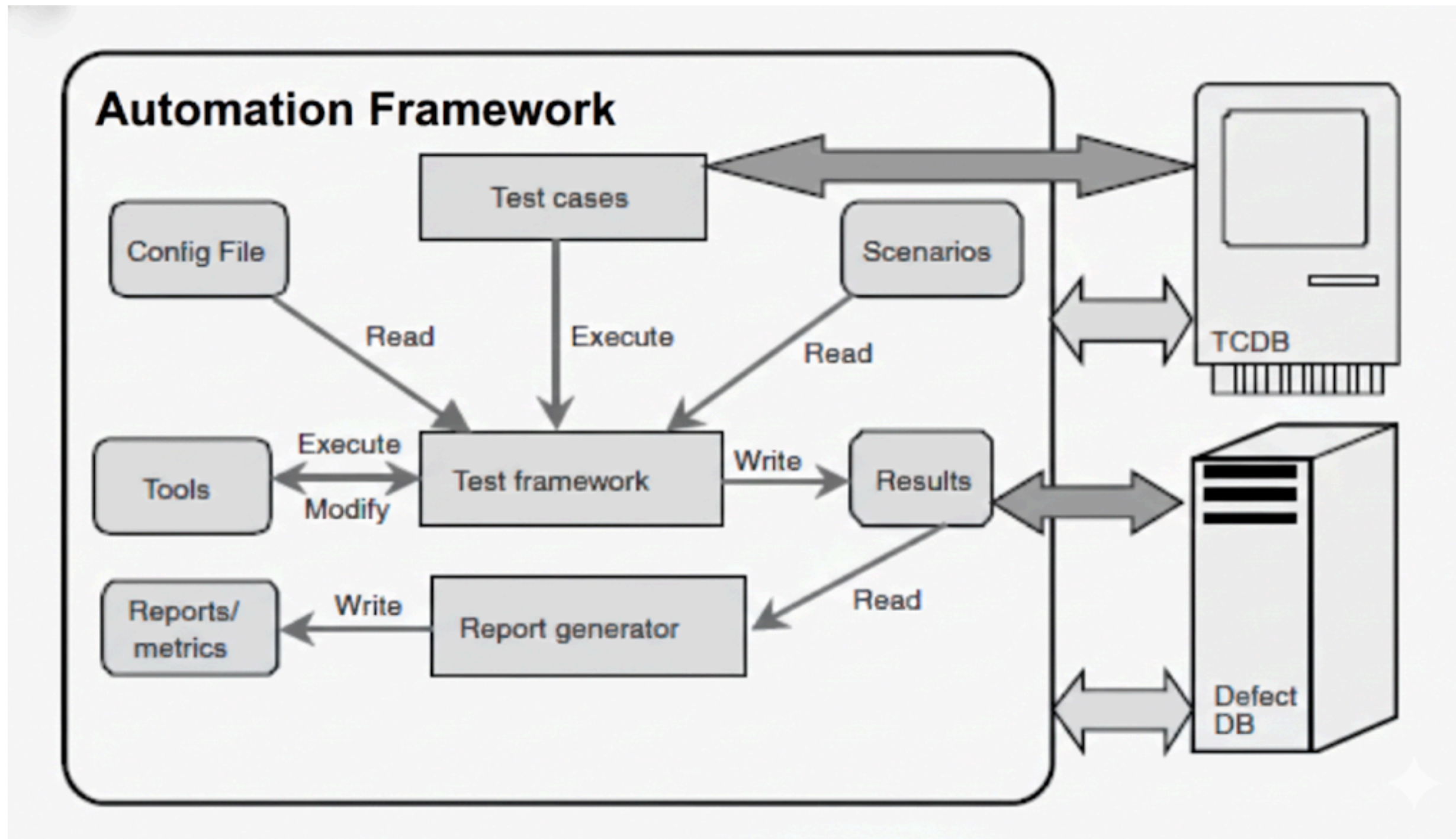
DESIGN AND ARCHITECTURE FOR AUTOMATION

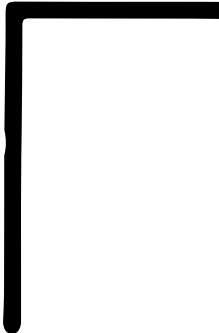
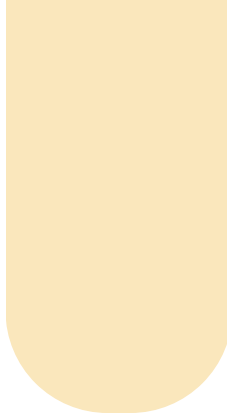
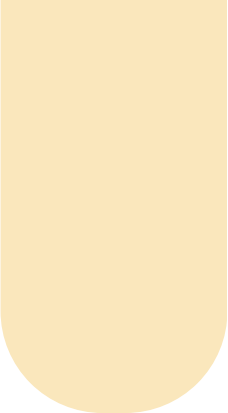
Framework for test automation



- The diagram shows a clear split between test cases and the framework/test tool. **Test cases define what a test should do**, while the **framework defines how to execute the tests**.
- **Test cases** contain the core product operations (e.g., login, file open). They describe the expected behavior and functionality to be verified.
- **Scenarios** describe different conditions under which tests run (load, performance, reliability, etc.). These are often user-defined.
- The **framework** sits in the middle and connects test cases with scenarios. It manages execution, applies scenarios, and runs the test cases accordingly.
- A **test suite** is created by combining multiple test cases with different scenarios through the framework, allowing the same operations to be tested in multiple ways.

Components of test automation





Just as in product development, automation architecture must represent all requirements through distinct modules and their interactions (both internal and external).

- The foundation consists of two external databases: a Test Case Database (TCDB) and a Defect Repository.
 - The test framework acts as the backbone that ties together the selection and execution of test cases.
- 
- 

External Modules

1.Test Case Database (TCDB)

- Stores all test cases, execution steps, and historical data (pass/fail status).
- While TCDB holds both manual and automated tests, the automation framework only interacts with the automated ones.

2.Defect Database (Defect DB)

- Contains details of all found defects (status, assignee, impact, etc.).
- Automation Benefit: The framework can automatically submit defects to this database during execution, unlike manual testing where engineers must submit them manually.

Scenario and Configuration File Modules

- **Scenario Module:** Describes how the process should be executed. (e.g., instructions to run a test for 48 hours or in a specific loop).
- **Configuration File:** Contains a set of variables used by the framework, tools, or specific test suites.
 - **Dynamic Execution:** Variables can be changed dynamically to test various input/output conditions and software states without changing the code.

Test Framework Module

- **Function:** Combines "what to execute" (test cases from TCDB) with "how to execute" (scenarios and config files).
- **Logic Center:** It contains the main logic for initiating, controlling, and monitoring all modules

Tools and Results Modules

- **Support Tools:** The framework invokes external utilities as needed, such as build tools (to compile test code) or simulators (e.g., IP Packet or Machine Simulators) to create the runtime environment.
- **Results & Archiving:**
 - Results must not be overwritten; a history must be archived.
 - Archiving allows the framework to perform smart selection, such as execute all test cases that failed in the previous run or selecting tests for regression cycles.

Report Generator and Metrics

- **Report Generator:** Automates the complex task of formatting raw results into readable documents.
- **Levels of Reporting:**
 - a. **Executive Reports:** High-level status for management.
 - b. **Technical Reports:** Moderate detail for lead engineers.
 - c. **Detailed/Debug Reports:** Granular detail for developers to fix specific failures.
- Reports are generated at various intervals (daily, weekly, monthly, or per milestone) to satisfy different stakeholders.
- All generated metrics and reports are stored in the **Reports/Metrics module** for future trend analysis.